`onComplete`. If the stream is not `completed` by then, the test fails.

Listing 16.1 External filter

```
class CPUNonBoundedSpec ... {
  ...

  implicit val ec =
    ExecutionContext.fromExecutor(Executors.newFixedThreadPool(1000))    ❶

  "calling to a fake external service" should {
    "be much faster" in {

      val init = System.currentTimeMillis

      val result: Future[Done] =
        Source(1 to 5000)
          .mapAsync(1000)(each => externalFilter(each))                  ❷
          .map(each => print1000th(each, init))                          ❸
          .run()

      Await.result(result, 6.seconds)
      Assert(result.isCompleted)
    }
  }
  ...
}
```

❶ Max number of futures in flight

❷ Calls to the external service

❸ Prints each thousandth element

With this setup, which calls a CPU-unbounded service, you can increase parallelism as much as the JVM has memory. Each thread consumes about 512 KB. The other constraints are the service you call and how many requests it can handle. In this example, the scheduler has little work scheduling 5,000 items. All 5,000 requests can be executed in less than 6 seconds.